

Techniques d'apprentissage
IFT 603 / IFT 712

Combinaison de modèles

Par
Pierre-Marc Jodoin
/
Hugo Larochelle

1

1

Pourquoi utiliser un seul modèle?

- Pourquoi utiliser un seul modèle?
 - un système combinant une multitude de modèles différents ne serait-il pas meilleur?
- En pratique, la réponse est presque toujours oui !
 - le résultat de la combinaison de plusieurs modèles est appelée **ensemble** ou **comité**

2

2

Pourquoi utiliser un seul modèle?

- La façon la plus simple d'obtenir M modèles est d'utiliser M algorithmes d'apprentissage différents :
 - POUR $i = 1$ à m
 - Entraîner un modèle $y_{w,i}(\vec{x})$ à l'aide du i -ième algo d'entraînement
- Retourner le **modèle ensemble** (ou comité)
 - Pour la **régression**
 - $y_{COM}(\vec{x}) = \frac{1}{m} \sum_{i=1}^m y_{w,i}(\vec{x})$
 - Pour la **classification**
 - **vote majoritaire**

3

3

Pourquoi utiliser un seul modèle?

- **Les M algorithmes pourraient être le même algorithme avec avec M sélections d'hyperparamètres différents.**
 - POUR $i = 1$ à m
 - Entraîner un modèle $y_{w,i}(\vec{x})$ à l'aide du i -ième algo d'entraînement
- Retourner le **modèle ensemble** (ou comité)
 - Pour la **régression**
 - $y_{COM}(\vec{x}) = \frac{1}{m} \sum_{i=1}^m y_{w,i}(\vec{x})$
 - Pour la **classification**
 - **vote majoritaire**

4

4

Pourquoi utiliser un seul modèle?

- Même avec **un seul algorithme et les mêmes hyperparamètres**, on peut améliorer sa performance à l'aide d'un ensemble.
 - **Bagging** : approprié pour combiner des modèles avec une **forte capacité**
 - **Boosting** : approprié pour combiner des modèles avec une **faible capacité**

5

5

Pourquoi utiliser un seul modèle?

- Même avec **un seul algorithme et les mêmes hyperparamètres**, on peut améliorer sa performance à l'aide d'un ensemble.
 - **Bagging** : approprié pour combiner des modèles avec une **forte capacité**
 - **Boosting** : approprié pour combiner des modèles avec une **faible capacité**

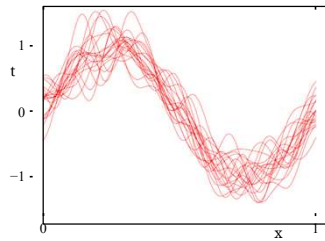
6

6

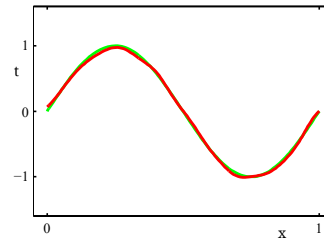
Bootstrap : réduction de la variance

- Régression polynomiale de degré 25

100 modèles entraînés sur
100 ensembles d'entraînement différents



Ensemble des 100 modèles vs. vrai modèle



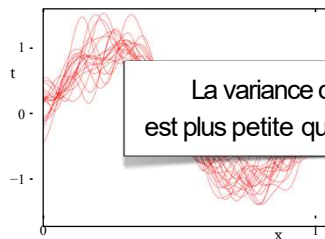
7

7

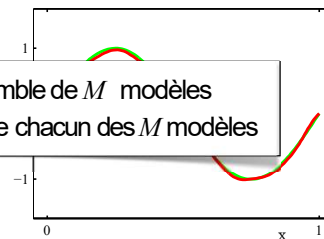
Bootstrap : réduction de la variance

- Régression polynomiale de degré 25

100 modèles entraînés sur
100 ensembles d'entraînement différents



Ensemble des 100 modèles vs. vrai modèle



La variance d'un ensemble de M modèles
est plus petite que celle de chacun des M modèles

8

8

Bootstrap

À part pour des données synthétiques, on ne peut pas générer des données sur demande.

Solution : **Bootstrapping**.

$$D_{bootstrap} = \{ \}$$

Pour N itérations

- Choisir aléatoirement un entier parmi $\{1, \dots, N\}$
- $D_{bootstrap} = D_{bootstrap} \cup \{(\vec{x}_n, t_n)\}$

retourner $D_{bootstrap}$

9

9

Bootstrap

À part pour des données synthétiques, on ne peut pas générer des données sur demande.

Solution : **Bootstrapping**.

$$D_{bootstrap} = \{ \}$$

Pour N itérations

- Choisir aléatoirement un entier parmi $\{1, \dots, N\}$
- $D_{bootstrap} = D_{bootstrap} \cup \{(\vec{x}_n, t_n)\}$

retourner $D_{bootstrap}$

échantillonne N exemples
avec remplacement

10

10

Bagging (Bootstrap AGGregatING)

1. Générer m ensembles d'entraînement avec **Bootstrap** $\{D_1, D_2, \dots, D_m\}$

2. **Entraîner** m modèles $y_{w,i}(\vec{x})$ (un pour chaque ensemble)

3. **Combiner** les m modèles

Régression: $y_{com}(\vec{x}) = \frac{1}{m} \sum_{i=1}^m y_{w,i}(\vec{x})$

Classification: $y_{com}(\vec{x}) = \text{sign} \left(\sum_{i=1}^m y_{w,i}(\vec{x}) \right)$ (2 classes)

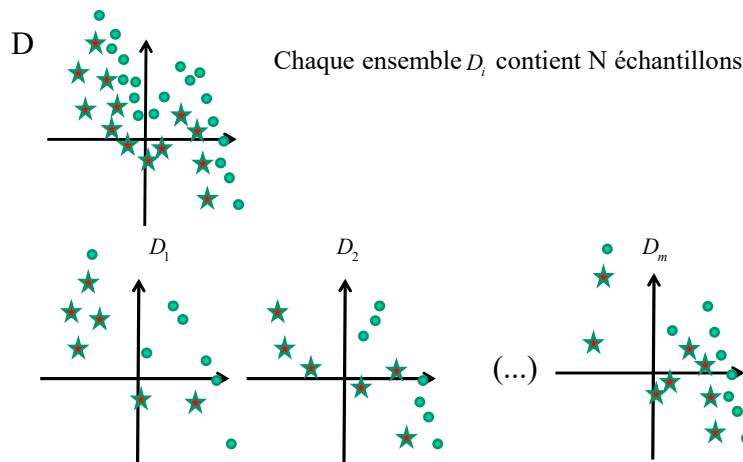
$y_{com}(\vec{x}) = \arg \max_c \left(\sum_{i=1}^m y_{w,i}(\vec{x}) \right)$ (K classes)

11

11

Illustration: classification 2Classes

1- **échantillonnage** avec replacement (*Bootstrap*)



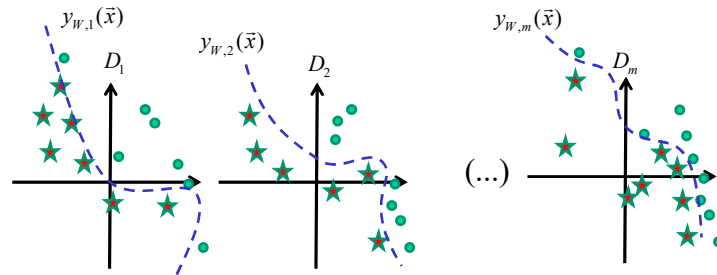
Note: un échantillon $\vec{x}_j$ peut apparaître plusieurs fois dans un même ensemble d'entraînement D_i

12

Illustration: classification 2Classes

2- Entraînement

Entraîner un modèle $y_{W,i}(\vec{x})$ (peut être perceptron, SVM, noyaux, etc.) sur chaque ensemble d'entraînement

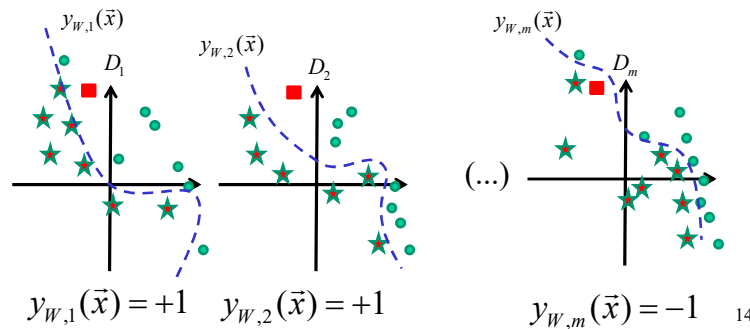


13

Illustration: classification 2Classes

3- Vote majoritaire

$$y_{com}(\vec{x}) = \text{sign}\left(\sum_{i=1}^m y_{W,i}(\vec{x})\right)$$



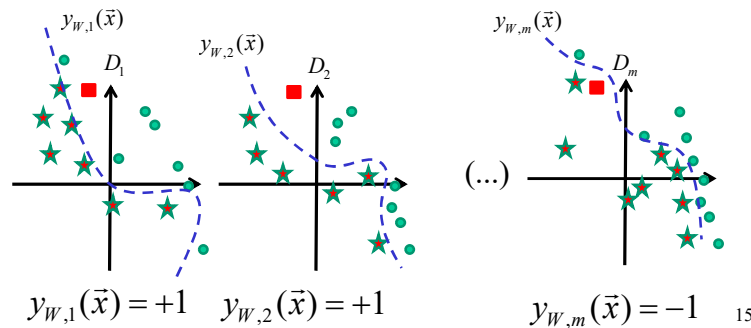
14

14

Illustration: classification 2Classes

3- **Vote majoritaire**

$$y_{com}(\vec{x}) = \text{sign}(1+1-1+1-1-1+\dots+1) = +1$$



15

Pourquoi utiliser une seul modèle?

- Même avec un seul algorithme sans hyper-paramètres, on peut améliorer sa performance à l'aide d'un ensemble.
 - **Bagging** : approprié pour combiner des modèles avec une **forte capacité**
 - **Boosting** : approprié pour combiner des modèles avec une **faible capacité**

17

17

AdaBoost

La méthode du boosting a pour objectif de **combiner plusieurs modèles faibles** (*weak learners*) afin d'obtenir un classifieur avec une plus grande capacité.

Trois (3) différences majeures avec le Bagging.

1. Implémente une **combinaison pondérée** de modèles. Ex. 2 classes

$$y_{com}(\vec{x}) = \text{sign} \left(\sum_{i=1}^m \alpha_i y_{w,i}(\vec{x}) \right)$$

2. **Pas de bootstrap**: chaque donnée $\vec{x}_n$ est utilisée pour entraîner les modèles
3. Les **données mal classées** par un modèle $y_{w,i}(\vec{x})$ auront **plus de poids** lors de l'entraînement du prochain modèle $y_{w,i+1}(\vec{x})$

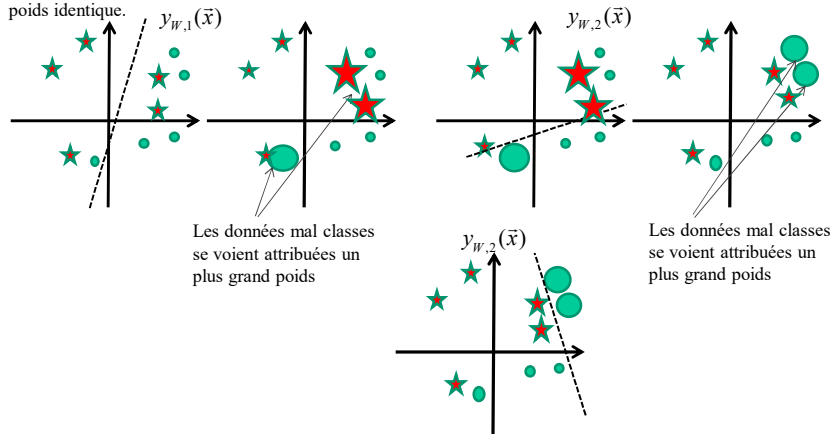
R. Schapire and Y. Freund **A decision theoretic generalization of on-line learning and an application to Boosting** Journal of Computer and System Sciences, 1997, 55: 119-139. 18

18

AdaBoost

(illustration 2 classes)

Le premier modèle est entraîné sur D, chaque donnée ayant un poids identique.



19

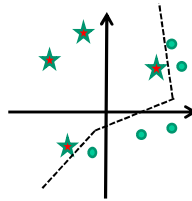
19

AdaBoost

(illustration 2 classes)

Le modèle combiné implémente une **combinaison pondérée** des 3 classifieurs faibles

$$y_{com}(\vec{x}) = \text{sign}(\alpha_1 y_{w,1}(\vec{x}) + \alpha_2 y_{w,2}(\vec{x}) + \alpha_3 y_{w,3}(\vec{x}))$$



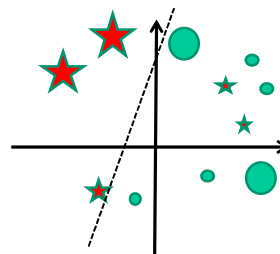
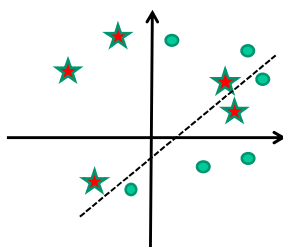
NOTE: plus un classifieur a une **exactitude élevée**, plus son α_i sera élevé. 20

20

AdaBoost

Idée fondamentale: chaque donnée $\vec{x}_i$ a un **poids** β_i

Lorsque les données ont toutes un **poids égale**, alors le **modèle faible** devient un classifieur linéaire comme un **perceptron** ou une regression logistique.



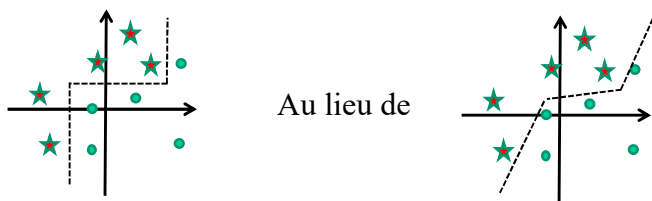
Poids égaux Vs poids non égaux

21

21

AdaBoost

Les modèles de type **stump** sont des classifieurs perpendiculaires à un axe
La combinaison de modèles **stump** mène à des frontières de décision “crénelées”



2 avantages: très **rapide** et permet d’identifier les **caractéristiques utiles**:

$$y_{com}(\vec{x}) = \text{sign}\left(\sum_{i=1}^m \alpha_i y_{W,i}(\vec{x})\right)$$

Les caractéristiques utiles sont celles pour lesquelles α_i est élevé

22

Adaboost

1- initialiser le poids des N données d’apprentissage: $\beta_i = \frac{1}{N} \quad \forall i$

POUR $i=1$ à m

2- Entraîner le modèle $y_{W,i}(\vec{x})$ avec les données D et les poids $\{\beta_1, \dots, \beta_N\}$

3- Calculer l’erreur d’entraînement: ε_i

$$\varepsilon_i = \frac{\sum_{\vec{x}_k \in \Psi} \beta_k}{\sum_{i=1}^N \beta_i} \quad \text{où } \Psi \text{ l'ensemble des données mal classées}$$

4- calculer $\alpha_i = \ln\left(\frac{1-\varepsilon_i}{\varepsilon_i}\right)$

5- mise à jour du poids des données mal classées par $y_{W,i}(\vec{x})$

$$\beta_n = \beta_n \exp\{\alpha_i\}$$

6- Normaliser les poids afin qu’ils somment à 1

$$\beta_k = \frac{\beta_k}{\sum_j \beta_j}$$

Le classifieur combiné: $y_{com}(\vec{x}) = \text{sign}\left(\sum_{i=1}^m \alpha_i y_{W,i}(\vec{x})\right)$

23

23

Adaboost

1- initialiser le poids des N données d'apprentissage: $\beta_i = \frac{1}{N} \quad \forall i$

POUR $i=1$ à m

2- Entraîner le modèle $y_{w,i}(\vec{x})$ avec les données D et les poids $\{\beta_1, \dots, \beta_N\}$

3- Calculer l'erreur d'entraînement: ε_i

$$\varepsilon_i = \frac{\sum_{\vec{x}_i \in \Psi} \beta_k}{\sum_{i=1}^N \beta_i} \quad \text{où } \Psi \text{ l'ensemble d'}$$

Plus un classifieur a une erreur faible, plus son poids α_i sera élevé

4- calculer $\alpha_i = \ln\left(\frac{1-\varepsilon_i}{\varepsilon_i}\right)$

5- mise à jour du poids des données mal classées par $y_{w,i}(\vec{x})$

$$\beta_n = \beta_n \exp\{\alpha_i\}$$

6- Normaliser les poids afin qu'ils somment à 1

$$\beta_k = \frac{\beta_k}{\sum_j \beta_j}$$

Le classifieur combiné: $y_{com}(\vec{x}) = \text{sign}\left(\sum_{i=1}^m \alpha_i y_{w,i}(\vec{x})\right)$

24

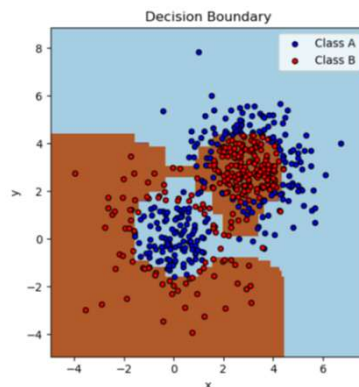
24

Sklearn

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
(...)
```

```
bdt = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1,
algorithm="SAMME",
n_estimators=200))
```

```
bdt.fit(X, y)
```



Classifieur « Stump »
Algo de la page précédente
200 modèles

25

25

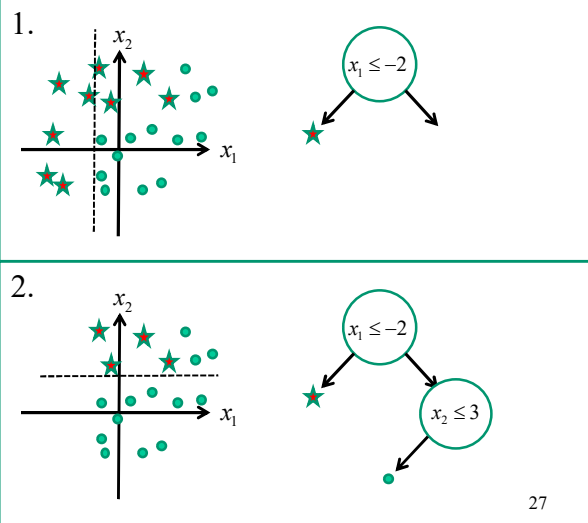
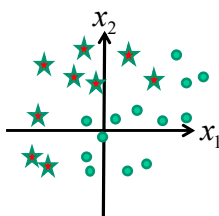
Arbres de décision

26

26

Arbres de décision

Exemple simple avec des classifieurs *stump*

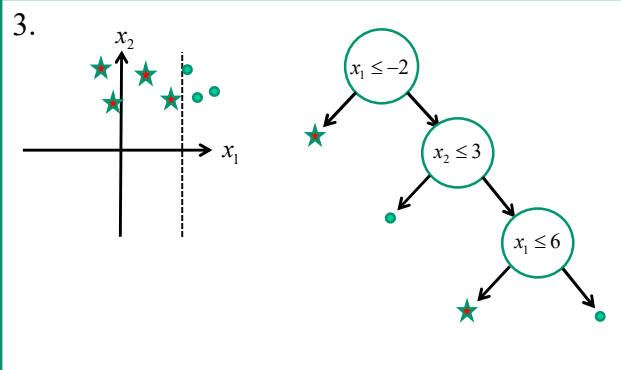
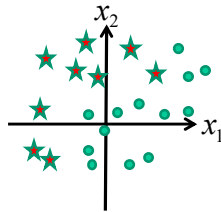


27

27

Arbres de décision

Exemple simple avec des classifieurs *stump*



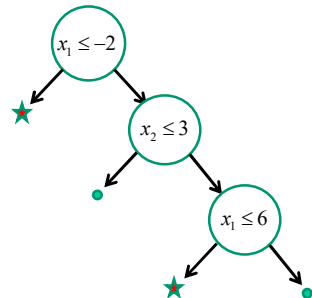
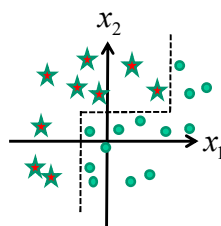
28

28

Arbres de décision

Exemple simple avec des classifieurs *stump*

Au final



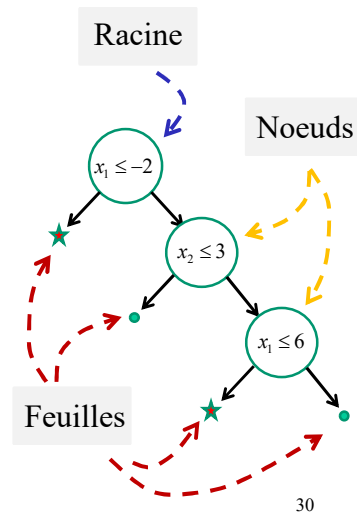
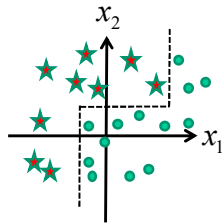
29

29

Arbres de décision

Exemple simple avec des classifieurs *stump*

Au final

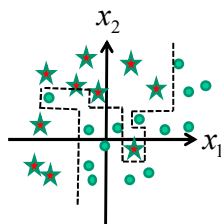


30

Arbres de décision

Le plus gros problème des arbres de décision est qu'ils ont tendance à **sur-apprendre**

Exemple avec 2
données aberrantes:



Grosse question : soit le noeud d'un arbre dont l'erreur d'entraînement n'est pas nulle
devons-nous le subdiviser ou non?

31

31

Arbres de décision

La décision de subdiviser ou non un noeud dépend d'une notion d' "impureté" d'un noeud

Si l'impureté d'un noeud est **élevée** → alors **on subdivise**
Si l'impureté d'un noeud est **faible** → alors **on ne subdivise pas**

Deux mesures d'impureté fréquemment utilisées

1. L'entropie

$$i(\text{node}) = - \sum_{j=1,2} P(c_j) \log_2(P(c_j))$$

où $P(c_j)$ est la proportion de données dans la classe c_j

2. L'indice de Gini

$$i(\text{node}) = 1 - \sum_{j=1,2} P^2(c_j)$$

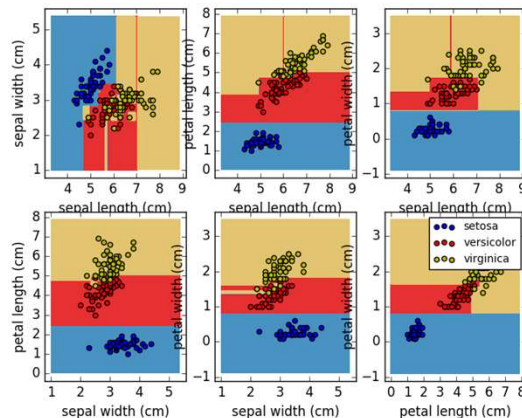
32

32

scikit-learn

```
from sklearn.tree import DecisionTreeClassifier
(...)
cfl = DecisionTreeClassifier(criterion='gini', splitter='best', max_depth=10,
                             min_impurity_split=0.0001)
cfl.fit(X,Y)
```

Decision surface of a decision tree using paired features



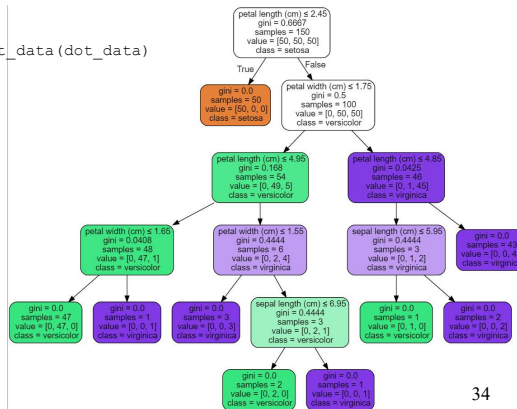
33

33

scikit-learn

```
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from IPython.display import Image
import pydotplus
(...)
cfl = DecisionTreeClassifier(criterion='gini', splitter='best', max_depth=10,
                             min_impurity_split=0.0001)
cfl.fit(X,Y)

dot_data = export_graphviz(cfl)
graph = pydotplus.graph_from_dot_data(dot_data)
Image(graph.create_png())
```

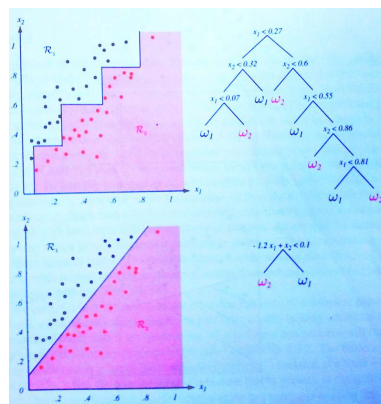


34

34

Arbres de décision

Les arbres de décision peuvent également inclure des classifieurs linéaires plus généraux



Credit, Duda Hart.

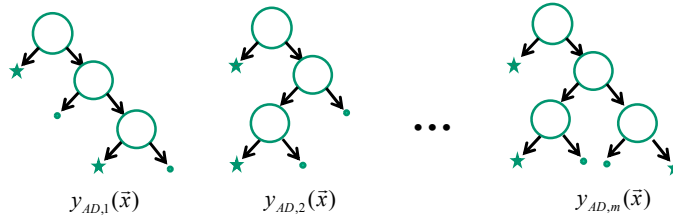
J Shotton, A Fitzgibbon, M Cook, T Sharp, M Finocchio, R Moore, A Kipman, and A Blake **Real-Time Human Pose Recognition in Parts from a Single Depth Image**

35

35

Forêt d'arbres décisionnels (FAD)

(Random forests)



En gros, les **FAD** ont pour but de combiner m **arbres de décision** à l'aide d'un **vote majoritaire**

$$y_{FAD}(\vec{x}) = \text{sign} \left(\sum_{i=1}^m y_{AD,i}(\vec{x}) \right) \quad (\text{segmentation 2 classes})$$

Très bonne référence!

https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm

36

36

Entraînement d'une FAD

FOR $i=1$ to m

Sélectionner au hasard N données $D_i = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$, $\vec{x}_i \in R^d$

*Sélectionner au hasard P dimensions $\Rightarrow \hat{D}_i = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$, $\vec{x}_i \in R^p, p < d$

Entraîner un arbre de décisions $y_{AD,i}(\vec{x})$ avec $\hat{D}_i$

Bootstrapping

Généralisation

Soit une nouvelle donnée $\vec{x} \in R^d$

FOR $i=1$ to m

$C_i = y_{AD,i}(\vec{x})$

$\text{class} = \text{VoteMajoritaire}(C_1, C_2, \dots, C_m)$

* Afin que les **erreurs** produites par les AD soient les **moins corrélées** possible.

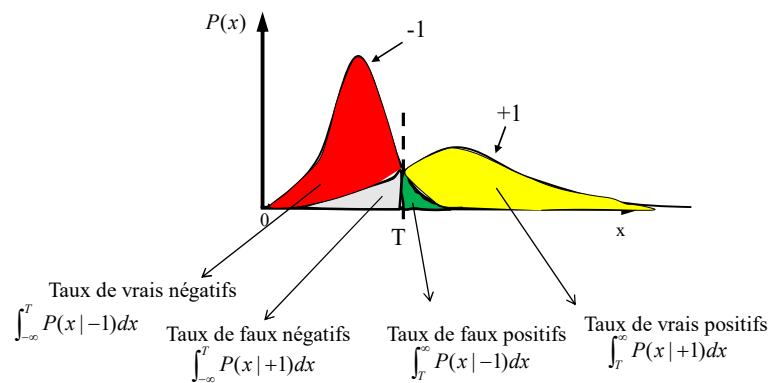
37

Métriques d'évaluation

38

38

Évaluation



Matrice de confusion

	Vérité terrain	
	Positifs	Négatifs
Sortie du modèle		
Positifs	TVP	TFP
Négatifs	TFN	TVN

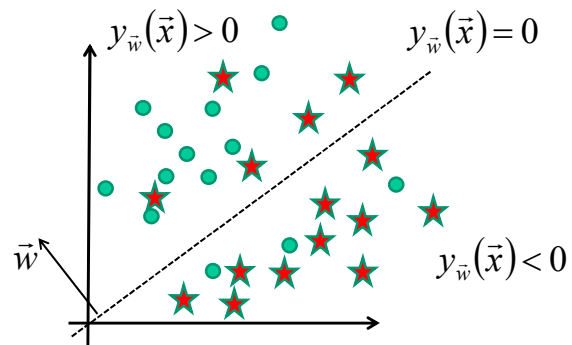
$$\begin{aligned} \text{TVN} + \text{TFP} &= 1 \\ \text{TVP} + \text{TFN} &= 1 \end{aligned}$$

$$\begin{aligned} \text{TVN} + \text{TFP} &= \text{prop des données dans la classe -1} \\ \text{TVP} + \text{TFN} &= \text{prop des données dans la classe +1} \end{aligned}$$

39

Évaluation

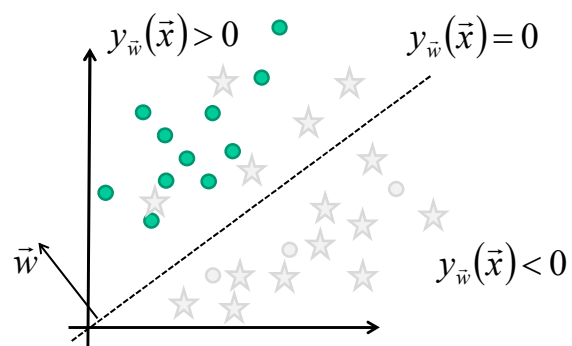
Validation empirique



40

Évaluation

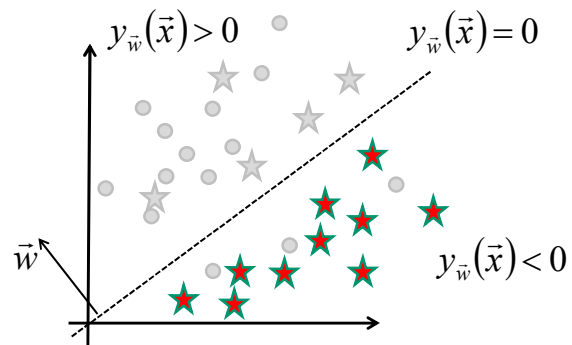
Vrais positifs



41

Évaluation

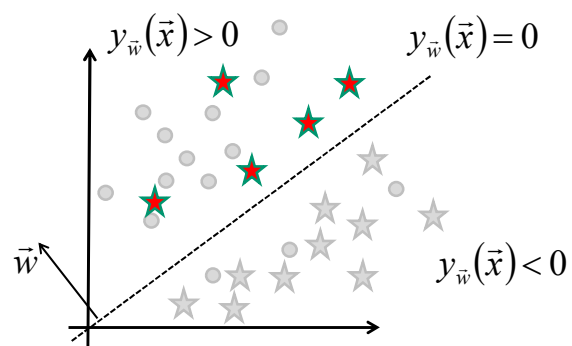
Vrais négatifs



42

Évaluation

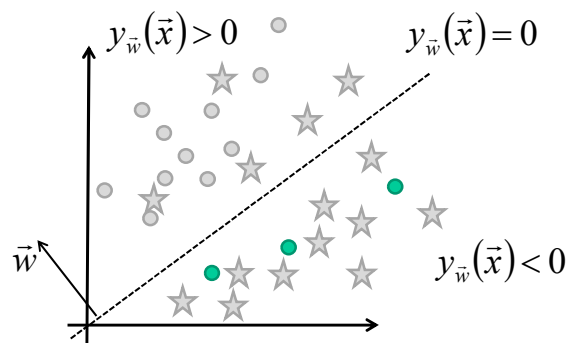
Faux positifs



43

Évaluation

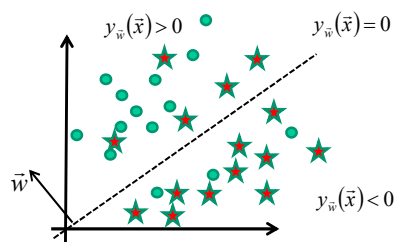
Faux négatifs



44

Évaluation

Validation empirique



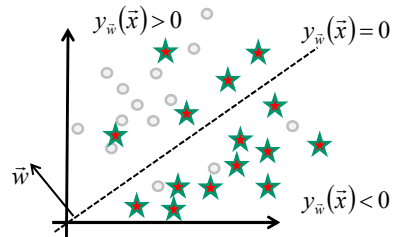
On obtient une matrice de confusion en comptant les données bien et mal classés

		Vérité terrain		
		Positifs	Négatifs	
Sortie du modèle	Positifs	VP = 11	FP = 5	VN + FP = 15 = nombre TOTAL de négatifs VP + FN = 14 = nombre TOTAL de positifs VP + FP = 16 = nombre d'éléments classés +1 FN + VN = 13 = nombre d'éléments classés -1
	Négatifs	FN = 3	VN = 10	

45

Évaluation

Validation empirique



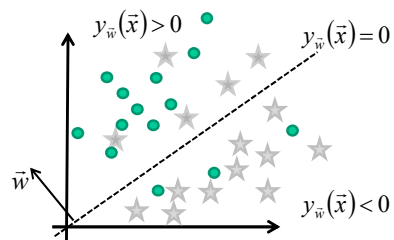
$$\text{Taux de faux positifs (TFP)} = \text{FP}/(\text{FP}+\text{VN}) = 5/15$$

46

46

Évaluation

Validation empirique



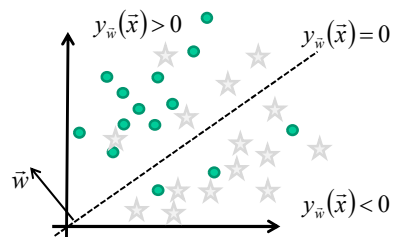
$$\text{Taux de faux négatifs (FNR)} = \text{FN}/(\text{FN}+\text{VP}) = 3/14$$

47

47

Évaluation

Validation empirique



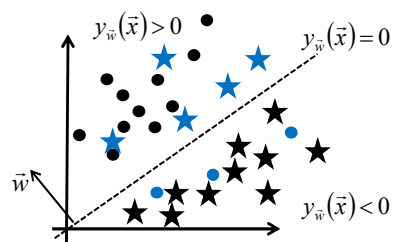
Rappel (*Recall* - **Re**) = $VP/(FN+VP)=1-TFN=11/14$

48

48

Évaluation

Validation empirique



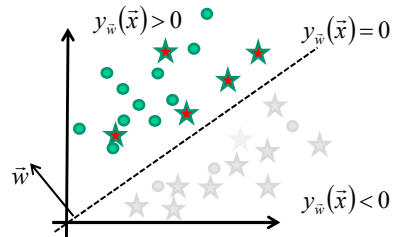
Justesse (« accuracy », **prop. de biens classés**) = $(TP+TN)/(FP+FN+TP+TN) = 21/29$

49

49

Évaluation

Validation empirique



$$\text{Précision (Pr)} = \text{VP}/(\text{VP}+\text{FP}) = 11/16$$

50

50

Évaluation

		Vérité terrain	
		Positifs	Négatifs
Sortie du modèle	Positifs	TP = 11	FP=5
	Négatifs	FN=3	TN=10

$\text{VN} + \text{FP} = 15 = \text{nombre de VRAIS négatifs}$
 $\text{VP} + \text{FN} = 14 = \text{nombre de VRAIS positifs}$
 $\text{VP} + \text{FP} = 16 = \text{nombre d'éléments classés +1}$
 $\text{VP} + \text{FN} = 13 = \text{nombre d'éléments classés -1}$

$$\text{Taux de faux positifs (TFP)} = \text{FP}/(\text{FP}+\text{VN}) = 5/15$$

$$\text{Taux de faux négatifs (TFN)} = \text{FN}/(\text{FN}+\text{VP}) = 3/14$$

$$\text{Spécificité (Sp)} = \text{VN}/(\text{FP}+\text{VN})=1-\text{FPR}=10/15$$

$$\text{Rappel (recall-Re)} = \text{VP}/(\text{VP}+\text{FN})=11/14$$

$$\text{Précision (Pr)} = \text{VP}/(\text{VP}+\text{FP}) = 11/16$$

$$\text{Justesse (Accuracy, prop. de biens classés)} = (\text{VP}+\text{VN})/(\text{FP}+\text{FN}+\text{VP}+\text{VN}) = 21/29$$

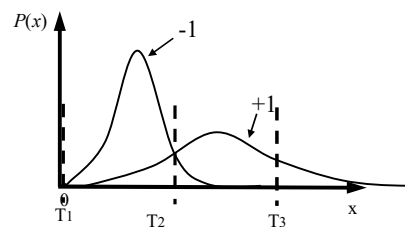
$$\text{F-measure} = 2 * (\text{Re} * \text{Pr})/(\text{Pr}+\text{Re})=0.73$$

51

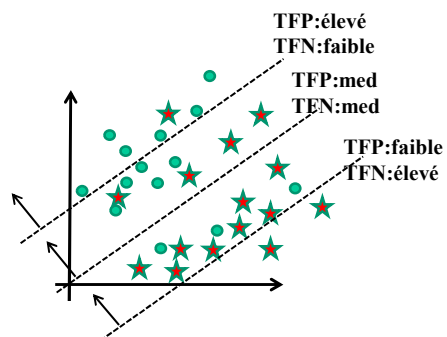
51

Évaluation

Différents seuils = différents résultats



TFP:élevé	TFP:med	TFP:faible
TFN:faible	TFN:med	TFN:élevé

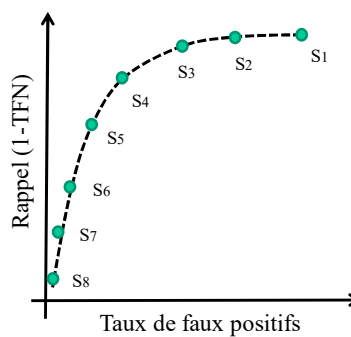
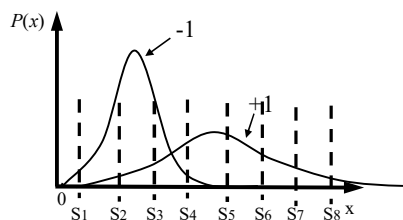


52

52

Évaluation

Courbe ROC : enregistrer le TFP et le TVP pour **différents seuils**



53

53

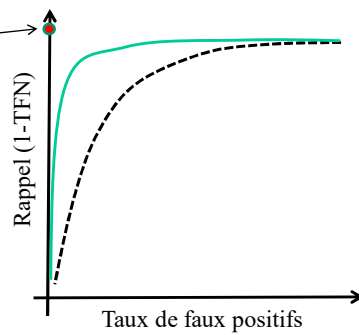
Validation

Courbes ROC: très bonne façon de comparer des méthodes

Quelle méthode est la meilleure?

Objectif ultime

La meilleure méthode
est celle avec
 $TFP=TFN=0$

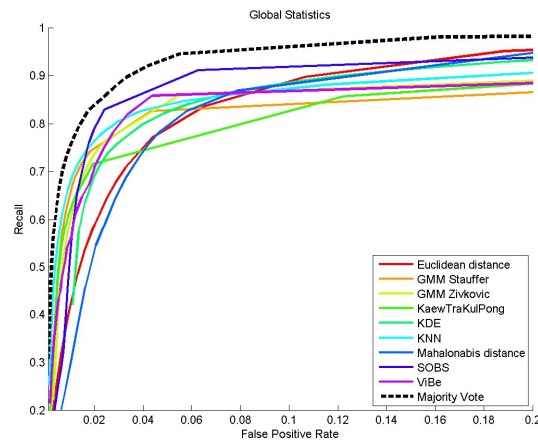


54

54

Évaluation

Exemple : 10 méthodes de détection de mouvement

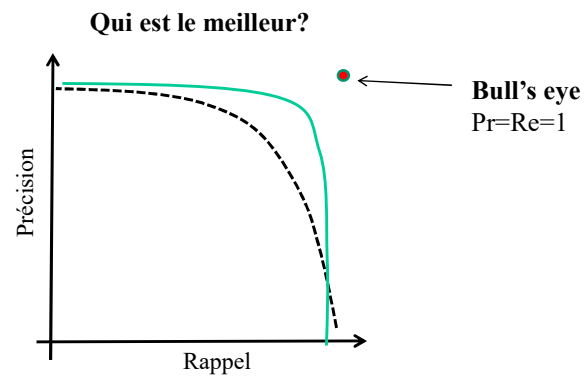


55

55

Évaluation

Courbe de précision-rappel : similaire à la courbe ROC



56

56